Campus Library — Video Presentation Script

CS543 · Web Services · GraphQL Hands-On Tutorial Walkthrough

Student Presenter: Bhupesh Kumar | **Tech Stack:** Node.js · Apollo Server · MySQL · Vanilla JS
Course: CS543 Web Services · Dr. Pramit Mazumdar, IIIT Vadodara | **Total Time:** ~4:30 Minutes

---

Step 1: Introduction & 3-Tier Architecture (0:00 - 0:40 | PDF Page 1)

■ **WHAT TO SHOW ON SCREEN:**
*Open VS Code showing project **Campus_Library** with files (schema.sql, db.js, server.js, public/) visible.*

■■ **WHAT TO SPEAK (EXACT SCRIPT):**
"Hello everyone and respected professor. Today I am demonstrating my implementation of the CS543 Web Services GraphQL hands-on tutorial.

Our goal is to build an end-to-end full-stack data application with three decoupled layers:
• **1. Frontend:** A lightweight web interface built with HTML, CSS, and JavaScript.
• **2. API Layer:** A GraphQL API powered by Node.js and Apollo Server running on port 4000.
• **3. Database Layer:** A MySQL database storing relational tables for authors and books.

As emphasized in the tutorial architecture, the browser never communicates with MySQL directly. It exclusively issues GraphQL queries and mutations via HTTP POST, and Apollo Server's resolvers execute SQL statements to interact with MySQL."

■ **Key Concept to Mention:** Mention that GraphQL operates over standard HTTP POST, so existing web tools and fetch() APIs work seamlessly.

Step 2: Relational Database Setup (schema.sql) (0:40 - 1:10 | Part 1 · Page 1-2)

■ **WHAT TO SHOW ON SCREEN:**
*Click and open **schema.sql** in VS Code. Highlight the table definitions and foreign key.*

■■ **WHAT TO SPEAK (EXACT SCRIPT):**
"In **schema.sql**, we define the relational schema inside the gqldemo database.

We create two tables:
• The **authors** table with primary key id, author name, and country.
• The **books** table with primary key id, title, publication year, and an author_id foreign key referencing authors(id).

This establishes a one-to-many relationship where one author can write multiple books. We seed the database with initial sample records: authors Asha Patel and Ravi Kumar, along with three initial books."

■ **Key Concept to Mention:** Relational foreign keys map naturally to GraphQL nested graphs: resolvers look up books matching author_id.

Step 3: Node Dependencies & Connection Pool (db.js) (1:10 - 1:40 | Part 2 & 3 · Page 2-3)

■ **WHAT TO SHOW ON SCREEN:**
*Show **package.json** briefly, then open **db.js**.*

■■ **WHAT TO SPEAK (EXACT SCRIPT):**
"In **package.json**, we installed the three required packages from Part 2: @apollo/server, graphql, and mysql2.

In **db.js**, we create an asynchronous MySQL connection pool using mysql2/promise. It connects to MySQL on localhost as user root using our password and targets the gqldemo database.

Using a connection pool is a best practice: it reuses open database connections efficiently and supports high-concurrency non-blocking I/O across our resolvers."

Step 4: GraphQL Schema & Resolvers (server.js) (1:40 - 2:30 | Part 4 · Page 3-4)

■ **WHAT TO SHOW ON SCREEN:**
*Open **server.js**. Scroll down through typeDefs, then the resolvers object.*

■■ **WHAT TO SPEAK (EXACT SCRIPT):**
"Now let's examine **server.js**, which contains the complete GraphQL API in three parts:

1. Schema Definition (typeDefs): We define the Author and Book types with bidirectional relationship fields, root queries for authors, author(id), and books, plus the addBook mutation.

2. The Resolvers:
• Root queries run straightforward SQL: SELECT * FROM authors.
• **Relationship Resolvers:** For Author.books, given a parent author, it executes SELECT * FROM books WHERE author_id = ?. For Book.author, it fetches the corresponding author.
• Notice that scalar fields like Book.title don't need resolvers — GraphQL resolves them automatically from the SQL column names.
• The addBook mutation executes an INSERT INTO books, retrieves the new record using result.insertId, and returns the full object.

3. Standalone Server: We initialize ApolloServer and bind it to port 4000 using startStandaloneServer."

■ **Key Concept to Mention:** Point out the 'Note — fields you do NOT write' from the tutorial: GraphQL maps column names to fields automatically.

Step 5: Starting the Server & Apollo Sandbox Testing (2:30 - 3:10 | Part 5 · Page 4-5)

■ **WHAT TO SHOW ON SCREEN:**
Open Terminal in VS Code, run node server.js. Then switch to browser at http://localhost:4000/ and run the nested authors query.

■■ **WHAT TO SPEAK (EXACT SCRIPT):**
"Let's start the server by executing node server.js in the terminal.
We see the confirmation message: GraphQL API ready at http://localhost:4000/.

Opening http://localhost:4000/ in the browser launches Apollo Sandbox Explorer.
I will paste the query from Part 5 of the tutorial, requesting every author's name, country, and their nested books with title and year.

When we click **Run**, Apollo Server fetches the data from MySQL and returns the exact hierarchical JSON shape requested — with zero over-fetching and zero under-fetching in a single round trip!"

Step 6: Frontend Web UI: Reading Authors & Books (3:10 - 3:50 | Part 6 & 7 · Page 5-6)

■ **WHAT TO SHOW ON SCREEN:**
*Switch to browser tab showing **index.html**. Refresh to show data loading cleanly.*

■■ **WHAT TO SPEAK (EXACT SCRIPT):**
"Now let's see how our web frontend consumes this API. In public/index.html, we have a clean interface styled with style.css.

Inside the script, the gql() helper is just a standard browser fetch() sending a POST request to http://localhost:4000/ with our query string.

When the page loads, the load() function calls the API and dynamically renders author cards for Asha Patel and Ravi Kumar, neatly listing each of their books with the publication year."

Step 7: Live Mutation: Adding a Book to MySQL (3:50 - 4:30 | Part 8 · Page 7)

■ **WHAT TO SHOW ON SCREEN:**
In 'Add a book' form, fill in Title: 'GraphQL for Beginners', Year: 2025, Select: 'Ravi Kumar'. Click 'Add book'. Show the new card item appearing.

■■ **WHAT TO SPEAK (EXACT SCRIPT):**
"Finally, let's demonstrate the write operation — the GraphQL mutation.

In the 'Add a book' form:
• I enter the title: *'GraphQL for Beginners'*.
• Enter year: *'2025'*.
• And select author: *'Ravi Kumar'*.

When I click **'Add book'**, the browser sends the addBook mutation with these variables. Apollo Server inserts the record into MySQL, and upon successful response, the page reloads the list.

As you can see, *'GraphQL for Beginners (2025)'* immediately appears under Ravi Kumar! The record is now permanently stored in MySQL."

Step 8: Summary & Conclusion (4:30 - 5:00 | Page 7-8 Recap)

■ **WHAT TO SHOW ON SCREEN:**
Switch back to VS Code, show the clean root project structure.

■■ **WHAT TO SPEAK (EXACT SCRIPT):**
"To summarize, we have fulfilled every requirement from the CS543 tutorial:
1. Created a MySQL relational database with authors and books.
2. Built a Node.js GraphQL API using Apollo Server on port 4000 with custom relationship resolvers.
3. Connected to MySQL using an asynchronous connection pool.
4. Built a responsive web frontend performing both read queries and write mutations.

Thank you very much for your time and attention!"